

Introduction à l'Apprentissage par Renforcement

Présenté par : HAUTOT Julien

ACIT - Actemium Clermont Ingenierie & Travaux

November 5, 2025

Plan de la présentation

- 1 Introduction générale
- 2 Fondements théoriques
- 3 Algorithmes tabulaires
- 4 Perspectives et applications
- 5 Conclusion

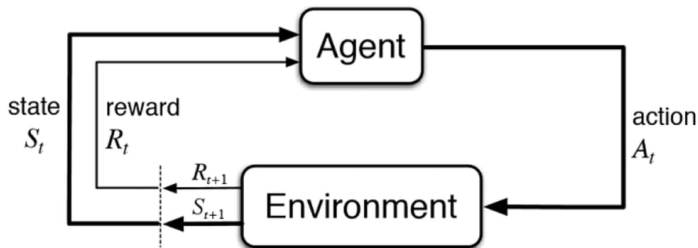
Pourquoi l'apprentissage par renforcement ?

- Les humains apprennent par **expérience** : essais, erreurs, récompenses.
- Le RL cherche à reproduire ce comportement chez une machine.
- Contrairement à l'apprentissage supervisé, il n'y a pas de "bon exemple" fourni.
- L'agent apprend à agir dans un environnement incertain pour **maximiser une récompense**.

Exemples

- AlphaGo : apprentissage du jeu de Go sans exemples humains.
- Robotique : apprentissage de la marche, de la manipulation d'objets.
- Recommandations : adapter les propositions selon les retours utilisateurs.

Cycle de décision de l'agent



- L'agent interagit en boucle avec l'environnement :

$$s_t \rightarrow a_t \rightarrow r_t, s_{t+1}$$

- Objectif : apprendre une **politique** $\pi(a|s)$ maximisant la récompense cumulative :

$$R = \sum_{t=0}^{\infty} \gamma^t r_t$$

Types d'environnement en RL

- **Discret vs Continu :**

- États et actions discrets.
- États et actions continus.

- **Déterministe vs Stochastique :**

- Déterministe : même action $\rightarrow$ même résultat.
- Stochastique : incertitude dans les transitions ou récompenses.

- **Épisodique vs Non-épisodique :**

- Épisodique : chaque épisode a une fin (ex. jeu, tâche finie).
- Non-épisodique : apprentissage continu sans fin claire.

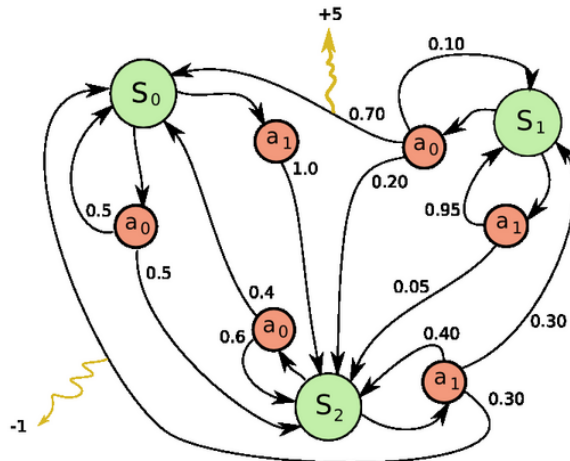
Programmation dynamique : l'ancêtre du RL

- Introduite par **Richard Bellman** dans les années 1950.
- Sert à résoudre des problèmes séquentiels où le modèle $P(s'|s, a)$ est connu.
- Principe : la solution optimale d'un grand problème se déduit des sous-problèmes optimaux.

Exemple intuitif

Trouver le chemin le plus court entre deux points — on calcule le meilleur choix local à chaque étape.

Processus de Décision de Markov (MDP)



Processus de Décision de Markov (MDP)

- Le RL repose sur la modélisation du problème comme un **MDP** :

$$(S, A, P, R, \gamma)$$

- S : états, A : actions, P : transitions, R : récompenses, γ : actualisation.
- Hypothèse de Markov : le futur dépend seulement du présent :

$$P(s_{t+1}|s_t, a_t, s_{t-1}, a_{t-1}, \dots) = P(s_{t+1}|s_t, a_t)$$

Valeur d'un état sous une politique π

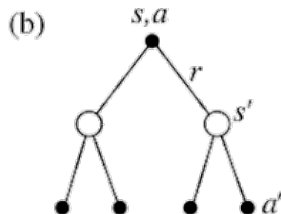
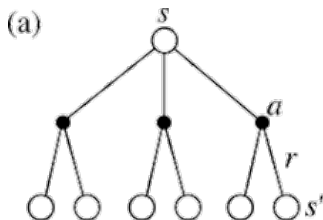
$$V^\pi(s) = \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t r_t \mid s_0 = s \right]$$

Valeur d'action (Q-valeur)

$$Q^\pi(s, a) = \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t r_t \mid s_0 = s, a_0 = a \right]$$

Idée clé : V^π évalue les **états**, Q^π évalue les **actions**.

Équation de Bellman : intuition graphique



(a) Bellman backup pour la fonction de valeur $V(s)$

(b) Bellman backup pour la fonction d'action $Q(s, a)$

Interprétation : La valeur d'un état ou d'un couple (s, a) dépend :

- de la récompense immédiate r ,
- et de la valeur des futurs états selon la politique optimale.

Équation de Bellman

Forme générale

$$V^\pi(s) = \sum_a \pi(a|s) \sum_{s'} P(s'|s, a) [R(s, a, s') + \gamma V^\pi(s')]$$

Forme optimale

$$V^*(s) = \max_a \sum_{s'} P(s'|s, a) [R(s, a, s') + \gamma V^*(s')]$$

① Forme théorique :

$$Q^*(s, a) = \sum_{s'} P(s'|s, a) [R(s, a, s') + \gamma \max_{a'} Q^*(s', a')]$$

② Forme échantillonnée :

$$Q(s, a) \leftarrow Q(s, a) + \alpha (r + \gamma \max_{a'} Q(s', a') - Q(s, a))$$

Intuition

L'agent met à jour ses croyances à partir de ses expériences.

Bootstrapping en RL

- Le **bootstrapping** consiste à estimer une valeur à partir d'une autre estimation.
- Exemple : mise à jour de $Q(s, a)$ avec $\max_{a'} Q(s', a')$.
- Utilisé dans :
 - Q-learning
 - SARSA
 - TD(0), TD(λ)

Attention

Le bootstrapping peut introduire de la variance et de l'instabilité.

Deux grandes familles d'apprentissage

- **Monte Carlo :**

- Mise à jour après chaque épisode complet.
- Utilise la somme réelle des récompenses.
- Pas de bootstrapping.

- **Temporal Difference (TD) :**

- Mise à jour à chaque étape.
- Utilise des estimations (bootstrapping).
- Plus rapide, mais plus instable.

TD(0) — Mise à jour immédiate

- TD(0) est une méthode d'apprentissage par différence temporelle.
- Mise à jour de la valeur d'un état après chaque transition.
- Utilise le bootstrapping : estimation à partir d'une autre estimation.
- Formule :

$$V(s_t) \leftarrow V(s_t) + \alpha [r_t + \gamma V(s_{t+1}) - V(s_t)]$$

- Rapide et efficace, mais sensible au bruit.

Intuition

On met à jour la valeur de l'état actuel en fonction de la récompense immédiate et de la valeur estimée du prochain état.

TD(λ) — Traces d'éligibilité et rôle de λ

- TD(λ) généralise TD(0) en tenant compte de plusieurs états passés.
- Chaque état visité reçoit une **trace d'éligibilité** $e_t(s)$ qui mesure son importance dans la mise à jour.
- Lorsqu'une récompense est reçue, tous les états précédents sont mis à jour proportionnellement à leur trace.
- Mise à jour :

$$V(s) \leftarrow V(s) + \alpha \cdot \delta_t \cdot e_t(s)$$

où :

- $\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$ est l'erreur TD
- $e_t(s)$ est la trace d'éligibilité de l'état s

TD(λ) — Traces d'éligibilité et rôle de λ

- La trace est mise à jour à chaque étape :

$$e_t(s) = \gamma \lambda \cdot e_{t-1}(s)$$

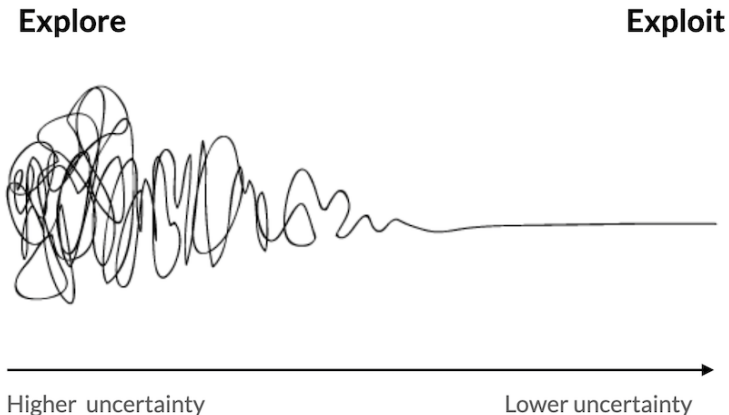
- Si s est l'état courant : $e_t(s) \leftarrow e_t(s) + 1$

Rôle de λ

$\lambda \in [0, 1]$ contrôle la durée de mémoire :

- $\lambda = 0$: TD(0), mise à jour immédiate.
- $\lambda = 1$: proche de Monte Carlo, mise à jour globale.

Exploration vs Exploitation



Exploration vs Exploitation

- Dilemme central : exploiter ce qu'on sait ou explorer pour apprendre mieux ?
- Stratégie simple : ϵ -**greedy**.

$$a_t = \begin{cases} \text{action aléatoire,} & \text{avec prob. } \epsilon \\ \arg \max_a Q(s_t, a), & \text{sinon} \end{cases}$$

Erreurs fréquentes et pièges classiques

- **Exploration insuffisante :**

- L'agent ne découvre pas les bonnes stratégies.
- Solution : ϵ -greedy, annealing, exploration guidée.

- **Récompenses mal définies :**

- Comportements inattendus ou non désirés.
- Importance du design du signal de récompense.

- **Environnement non stationnaire :**

- Les règles changent au cours du temps.
- Nécessite des algorithmes adaptatifs ou robustes.

Q-Learning : apprentissage hors politique

- **Off-policy** : apprend la politique optimale indépendamment de celle utilisée.
- Mise à jour :

$$Q(s, a) \leftarrow Q(s, a) + \alpha[r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$$

- Converge vers Q^* si l'exploration est suffisante.

SARSA : apprentissage sur politique

- **On-policy** : suit et apprend la politique qu'il exécute.
- Mise à jour :

$$Q(s, a) \leftarrow Q(s, a) + \alpha[r + \gamma Q(s', a') - Q(s, a)]$$

- Plus prudent, plus stable dans les environnements stochastiques.

Différence clé

Q-learning apprend la politique optimale. SARSA apprend la politique effectivement suivie.

Passons à la pratique !

- Nous allons appliquer les algorithmes tabulaires à deux environnements OpenAI Gym :
 - **Taxi-v3** : Q-learning
 - **MountainCar-v0** : SARSA

Défis et extensions du RL

- **Grand espace d'états/actions** : tabulaire devient impossible → Deep RL.
- **Récompenses rares ou bruitées** : exploration plus difficile.
- **Stabilité de l'apprentissage** : variance, divergence, etc.

Solutions modernes

- Deep Q-Networks (DQN)
- Actor-Critic, PPO, A3C
- Apprentissage multi-agents

Applications emblématiques

- **Jeux** : AlphaGo, Dota 2, StarCraft II.
- **Robotique** : navigation, manipulation d'objets.
- **Finance** : allocation de portefeuille.
- **Santé** : optimisation de traitements personnalisés.
- **Recommandation** : adaptation dynamique selon le retour utilisateur.

Conclusion

- Le RL repose sur les principes de la programmation dynamique et des MDP.
- Q-learning et SARSA sont les piliers historiques du RL tabulaire.
- Les développements modernes (Deep RL) étendent ces idées à des contextes complexes.

Message final

"Le RL, c'est l'art d'apprendre à agir — pas à prédire."